Lecture 6 Application in Model Checking

- Transition systems and traces
- Safety properties and invariants
- Model-checking regular safety properties

Model checking

- Automated model-based verification and debugging technique
  - model of system = Kripke structure $\approx$ labeled transition system
  - properties expressed in temporal logic like LTL or CTL
  - provides counterexamples in case of property refutation
- Various striking examples
  - Needham-Schroeder security protocol, storm surge barrier, C code
- 2008: Pioneers awarded prestigious ACM Turing Award





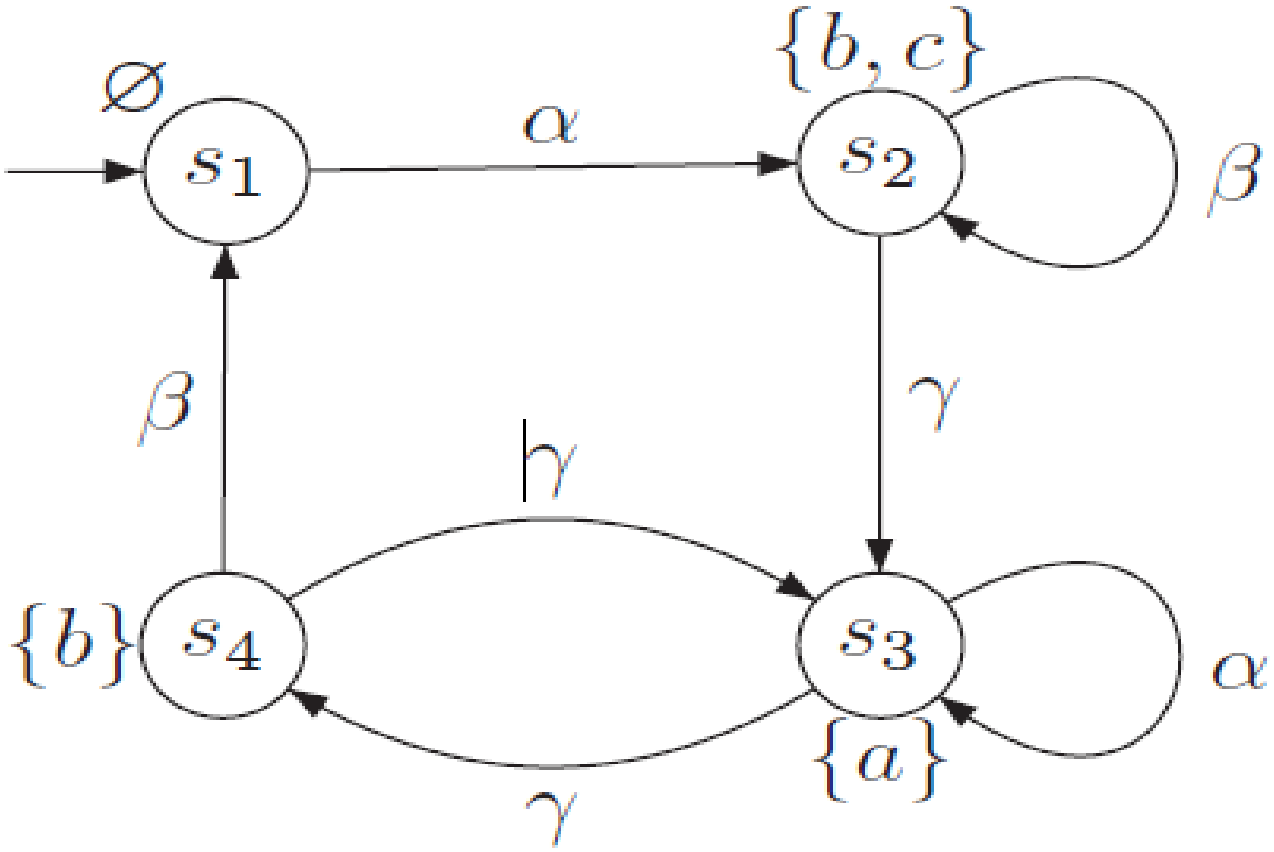
Transition system (TS)

A *transition system* TS is a tuple $(S, Act, \rightarrow, I, AP, L)$ where

- S is a set of states,
- Act is a set of actions,
- $\longrightarrow \subseteq S \times Act \times S$ is a transition relation,
- $I \subseteq S$ is a set of initial states,
- AP is a set of atomic propositions, and
- $L : S \rightarrow 2^{AP}$ is a labeling function.

Example

TS : AP={a,b,c}



Some notions

Let $TS = (S, Act, \rightarrow, I, AP, L)$ be a transition system. For $s \in S$ and $\alpha \in Act$, the set of direct α-*successors* of s is defined as:

$$Post(s, \alpha) = \left\{ s' \in S \mid s \xrightarrow{\alpha} s' \right\}, \quad Post(s) = \bigcup_{\alpha \in Act} Post(s, \alpha).$$

The set of α-*predecessors* of s is defined by:

$$Pre(s, \alpha) = \left\{ s' \in S \mid s' \xrightarrow{\alpha} s \right\}, \quad Pre(s) = \bigcup_{\alpha \in Act} Pre(s, \alpha).$$

State s in transition system TS is called *terminal* if and only if $Post(s) = \varnothing$. ■

Some more notions

- A finite path fragment $\hat{\pi}$ of TS is a finite state sequence $s_0 s_1 \cdots s_n$ such that $s_i \in Post(s_{i-1})$ for all $0 < i \leq n$, where $n \geq 0$. An infinite path fragment π is an infinite state sequence such that $s_i \in Post(s_{i-1})$ for all i>0
- A maximal path fragment is either a finite path fragment that ends in a terminal state or an infinite path fragment. A path fragment is called initial if it starts in an initial state.
- A path is an initial, maximal path fragment.

Path(TS) denotes the set of all paths in TS, and $Path_{fin}(TS)$ The set of all initial finite path fragments of TS.

Traces

- We consider states instead actions; in particular, since states are not observable, we focus on their labeling atomic propositions.
- For example, for an execution of the form

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots$$

we consider the sequences $L(s_1)L(s_2)L(s_3)\ldots$. Such sequences are called traces.

The traces of a transition system are thus words or strings over the alphabet 2^{AP}

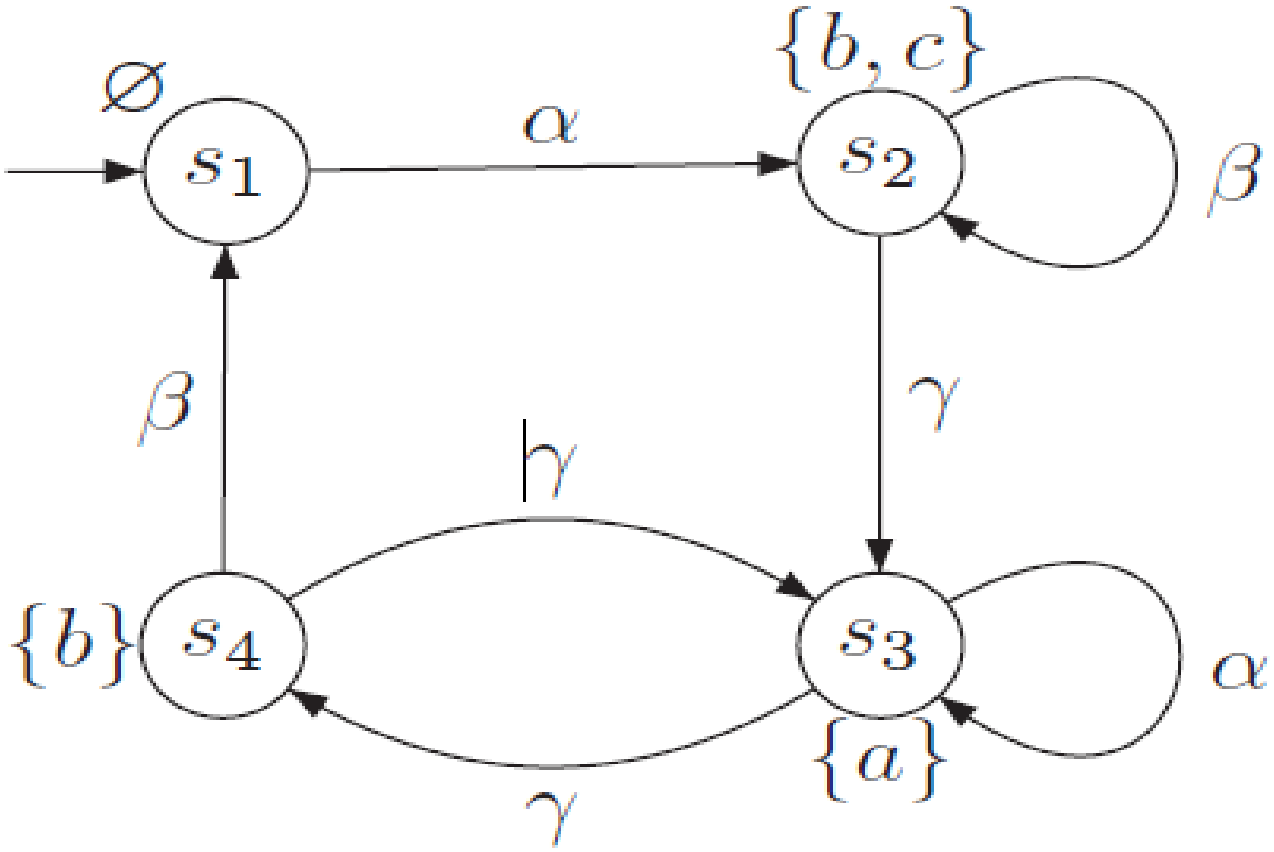
The alphabet is NOT AP!!!!

Traces

- Let transition system $TS = (S, Act, \rightarrow, I, AP, L)$ without terminal states
- The *trace* of path fragment $\pi = s_0\, s_1 \ldots$ is $trace(\pi) = L(s_0)\, L(s_1) \ldots$
  - the trace of $\widehat{\pi} = s_0\, s_1 \ldots s_n$ is $trace(\widehat{\pi}) = L(s_0)\, L(s_1) \ldots L(s_n)$.
- The set of traces of a set Π of paths: $trace(\Pi) = \{\, trace(\pi) \mid \pi \in \Pi \,\}$
- $Traces(s) = trace(Paths(s))$ $\qquad$ $Traces(TS) = \bigcup_{s \in I} Traces(s)$
- $Traces_{fin}(s) = trace(Paths_{fin}(s))$ $\qquad$ $Traces_{fin}(TS) = \bigcup_{s \in I} Traces_{fin}(s)$

Example

TS : AP={a,b,c}



Linear-time (LT) properties

- Linear-time properties specify the traces that a TS must exhibit
  - LT-property specifies the admissible behaviour of the system
  - later, a logical formalism will be introduced for specifying LT properties
- A *linear-time property* (LT property) over *AP* is a subset of $(2^{AP})^{\omega}$
  - finite words are not needed, as it is assumed that there are no terminal states
- *TS* (over *AP*) *satisfies* LT-property P (over *AP*):

$$TS \models P \quad \text{if and only if} \quad Traces(TS) \subseteq P$$

Example

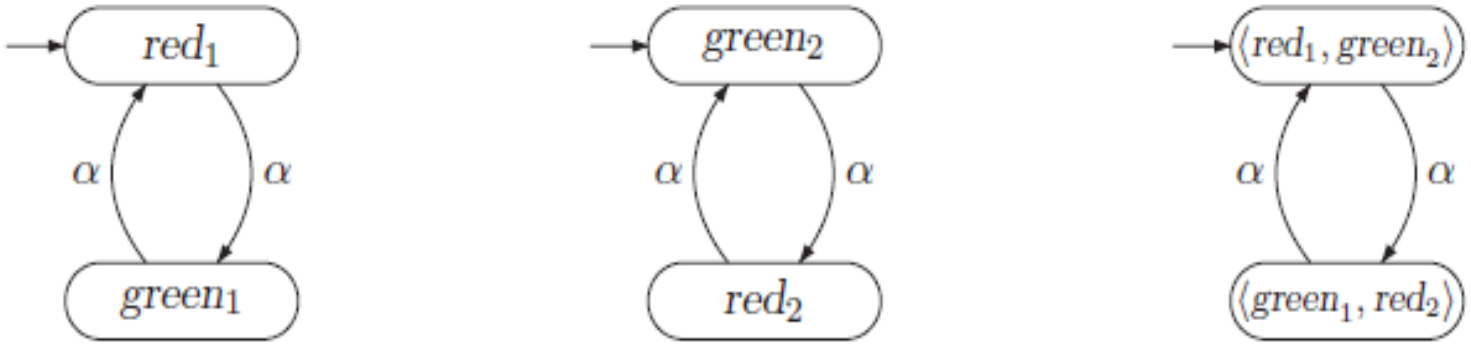


Figure 3.7: Two fully synchronized traffic lights (left and middle) and their parallel composition (right).

We consider two LT properties of these traffic lights and give some example words that are contained by such properties. First, consider the property P that states:

"The first traffic light is infinitely often green".

Propositional (Boolean) Logic

- Boolean operations: negation, conjunction (AND), disjunction (OR)

$$\varphi := p \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \neg\varphi$$

where p is an atomic proposition

- Equivalent expressions

$$(P \rightarrow Q) = (\neg P \vee Q)$$

$$(P \vee Q) = \neg(\neg P \wedge \neg Q)$$

- The distributive laws

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R),$$

$$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$$

Satisfaction

- Assume that $A \in 2^{AP}$

  φ is satified at A if the conjunction of atomic propositions at A is the positive conjuncts in the DNF of φ.

  - For example, assume that AP={a, b,c}, then

$$\{a,b\} \models b \wedge \neg c$$

So φ is satisfied at a state s if $L(s) \models \varphi$.

Invariants

An LT property P_{inv} over AP is an *invariant* if there is a propositional logic formula[5] Φ over AP such that

$$P_{inv} = \left\{ A_0 A_1 A_2 \ldots \in (2^{AP})^\omega \mid \forall j \geqslant 0.\ A_j \models \Phi \right\}.$$

Φ is called an invariant condition (or state condition) of P_{inv}. ■

Note that

$$\begin{aligned} TS \models P_{inv} \quad & \text{iff} \quad trace(\pi) \in P_{inv} \text{ for all paths } \pi \text{ in } TS \\ & \text{iff} \quad L(s) \models \Phi \text{ for all states } s \text{ that belong to a path of } TS \\ & \text{iff} \quad L(s) \models \Phi \text{ for all states } s \in Reach(TS). \end{aligned}$$

Invariant is a state property!

Algorithm 3 Naïve invariant checking by forward depth-first search

Input: finite transition system TS and propositional formula Φ
Output: true if TS satisfies the invariant "always Φ", otherwise false

---

```
set of state R := ∅;                          (* the set of visited states *)
stack of state U := ε;                        (* the empty stack *)
bool b := true;                               (* all states in R satisfy Φ *)
for all s ∈ I do
  if s ∉ R then
    visit(s)                                  (* perform a dfs for each unvisited initial state *)
  fi
od
return b
```

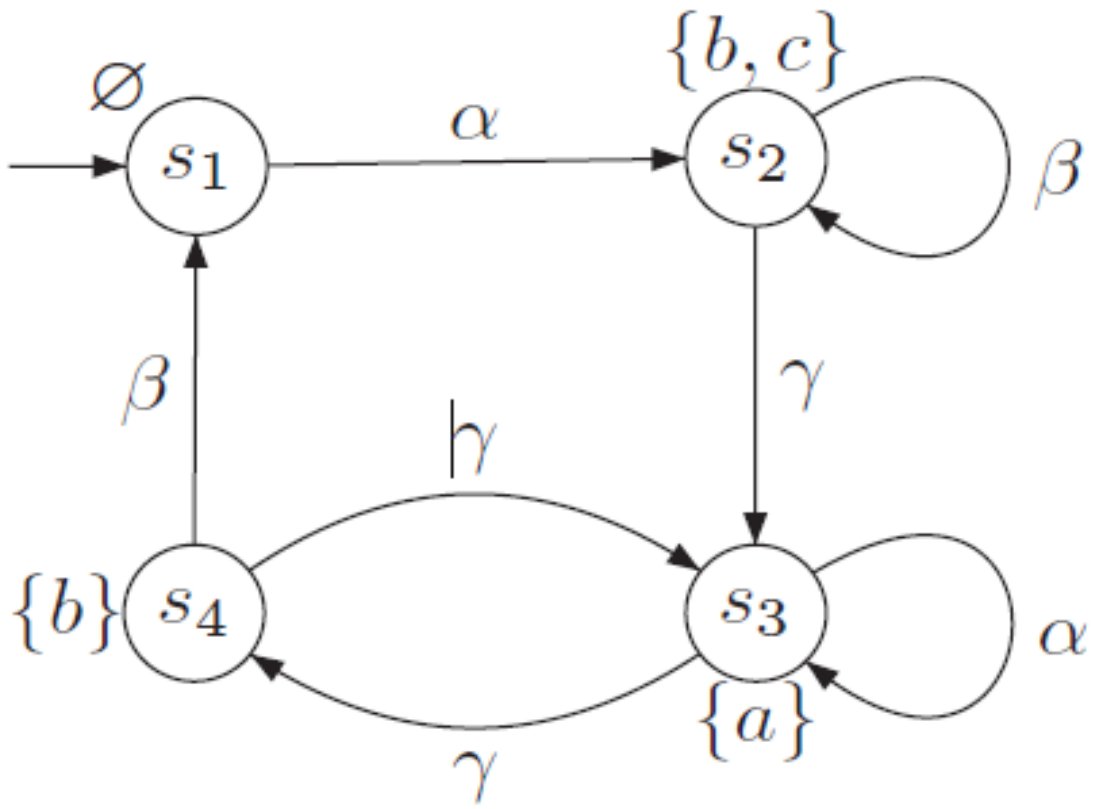
```
procedure visit (state s)
  push(s, U);                                 (* push s on the stack *)
  R := R ∪ { s };                             (* mark s as reachable *)
  repeat
    s' := top(U);
    if Post(s') ⊆ R then
      pop(U);
      b := b ∧ (s' ⊨ Φ);                      (* check validity of Φ in s' *)
    else
      let s'' ∈ Post(s') \ R
      push(s'', U);
      R := R ∪ { s'' };                       (* state s'' is a new reachable state *)
    fi
  until (U = ε)
endproc
```

Take **P_{inv} =a /\ b/\ c**

R: the set of reachable states
U: to be checked whether invariant is satisfied
b: whetherthe invariant is sati at all reachable states

Example

$TS:$



Algorithm 4 Invariant checking by forward depth-first search

Input: finite transition system TS and propositional formula Φ
Output: "yes" if $TS \models$ "always Φ", otherwise "no" plus a counterexample

---

set of states $R := \varnothing$; (* the set of reachable states *)
stack of states $U := \varepsilon$; (* the empty stack *)
bool $b :=$ true; (* all states in R satisfy Φ *)
while $(I \setminus R \neq \varnothing \ \wedge \ b)$ **do**
  let $s \in I \setminus R$; (* chocse an arbitrary initial state not in R *)
  visit(s); (* perform a DFS for each unvisited initial state *)
od
if b **then**
  return("yes") (* $TS \models$ "always Φ" *)
else
  return("no", reverse(U)) (* counterexample arises from the stack content *)
fi

---

procedure visit (state s)
  $push(s,U)$; (* push s on the stack *)
  $R := R \cup \{ s \}$; (* mark s as reachable *)
  repeat
    $s' := top(U)$;
    if $Post(s') \subseteq R$ **then**
      $pop(U)$;
      $b := b \ \wedge \ (s' \models \Phi)$; (* check validity of Φ in s' *)
    else
      let $s'' \in Post(s') \setminus R$
      $push(s'',U)$;
      $R := R \cup \{ s'' \}$; (* state s'' is a new reachable state *)
    fi
  until $((U = \varepsilon) \ \vee \ \neg b)$
endproc

Safety properties and bad prefixes

- An LT property P_{safe} over AP is called a safety property if for all words $\sigma \in (2^{AP})^{\omega} \setminus P_{safe}$ there exists a finite prefx $\hat{\sigma}$ of σ such that

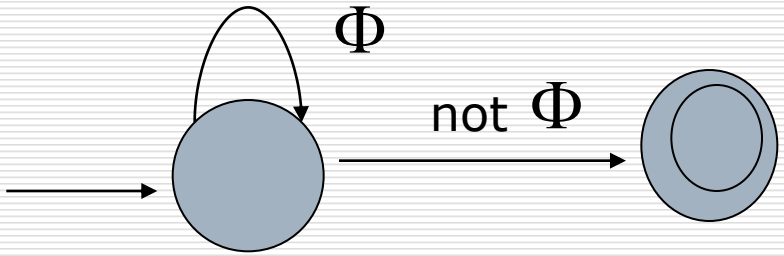
$$P_{safe} \cap \{\sigma' \in (2^{AP})^{\omega} : \hat{\sigma} \prec \sigma'\} = \emptyset$$

Minimal bad prefixes

- Any invariant is a safety property. For a propositional formula Φ and its invariant P_{inv} , all finite words of the form

$$A_0 A_1 A_2 \cdots A_n \in (2^{AP})^*$$

with $A_0 \models \Phi, A_1 \models \Phi, \cdots, A_n \not\models \Phi$ constitute the minimal bad prefixes for P_{inv}



- But the converse is not true.

Characterization of safety properties

- For transition system TS without terminal states and safety property P_{safe}

$$TS \models P_{safe} \Leftrightarrow Trace_{fin}(TS) \cap (Min)Bad\,\mathrm{Pr}ef\,(P_{safe}) = \phi$$

A safety property for a traffic light

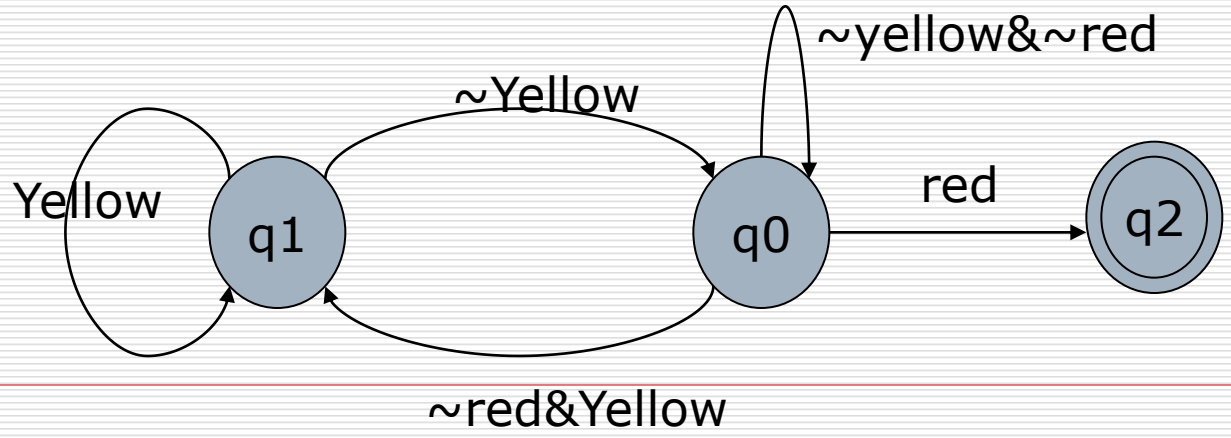
- The atomic propositions are red, green, and yellow. They serve to mark the states describing a phase.
- Specify the following properties:
  1. "always at least one of the lights is on"
  2. "It is never the case that two lights are switched on at the same time"
  3. "A red phase must be preceded immediately by a yellow phase"

- Are these properties safe? If yes, what are their corresponding (minimal) bad prefixes?

Regular safety properties

- Safety property P_{safe} over AP is called regular if its set of bad prefixes constitute a regular language over 2^{AP}
- Every invariant is a regular safety property.

Property: "A red phase must be preceded immediately by a yellow phase"



Note: this safety is not an invaraint.

Product of Transition system and NFA

- Let $TS = (S, Act, \rightarrow, I, AP, L)$ be a transition system without terminal states and $A = (Q, \Sigma, \delta, Q_0, F)$ an NFA with the alphabet $\Sigma = 2^{AP}$

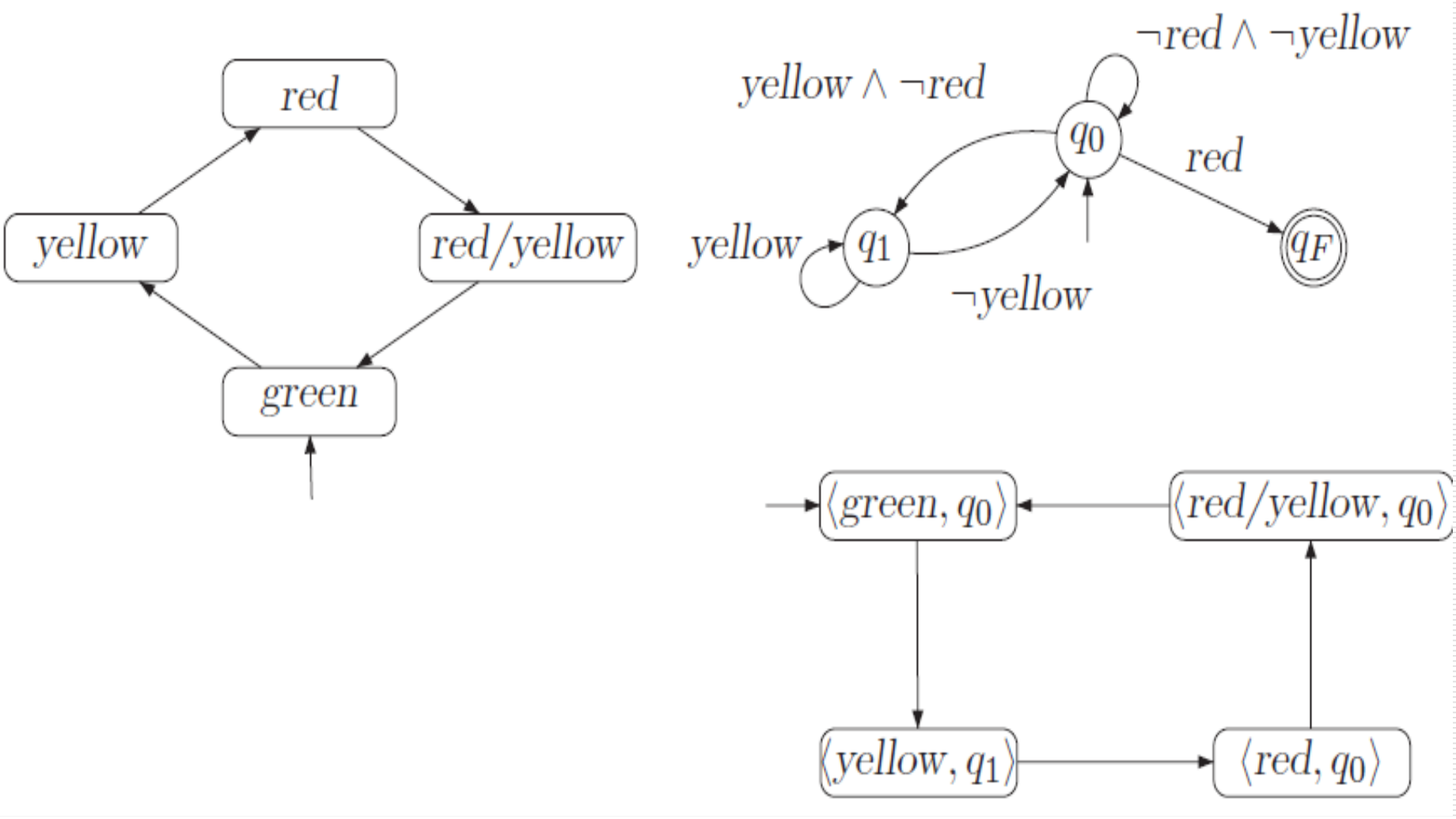
$$TS \otimes \mathcal{A} \;=\; (S', Act, \rightarrow', I', AP', L')$$

- $S' = S \times Q$,
- $\rightarrow'$ is the smallest relation defined by the rule

$$\frac{s \xrightarrow{\alpha} t \;\wedge\; q \xrightarrow{L(t)} p}{\langle s, q\rangle \xrightarrow{\alpha}' \langle t, p\rangle},$$

- $I' = \{\, \langle s_0, q\rangle \mid s_0 \in I \;\wedge\; \exists q_0 \in Q_0.\; q_0 \xrightarrow{L(s_0)} q \,\}$,
- $AP' = Q$, and
- $L' : S \times Q \to 2^Q$ is given by $L'(\langle s, q\rangle) \;=\; \{\, q \,\}$.

A product TS for German traffic light



What is the role of NFA in the product

- A sequence fragment in TS $\pi = s_0 s_1 \cdots s_n$ can be extended to a path fragment

$$\langle s_0, q_1 \rangle \langle s_1, q_2 \rangle \cdots \langle s_n, q_{n+1} \rangle$$

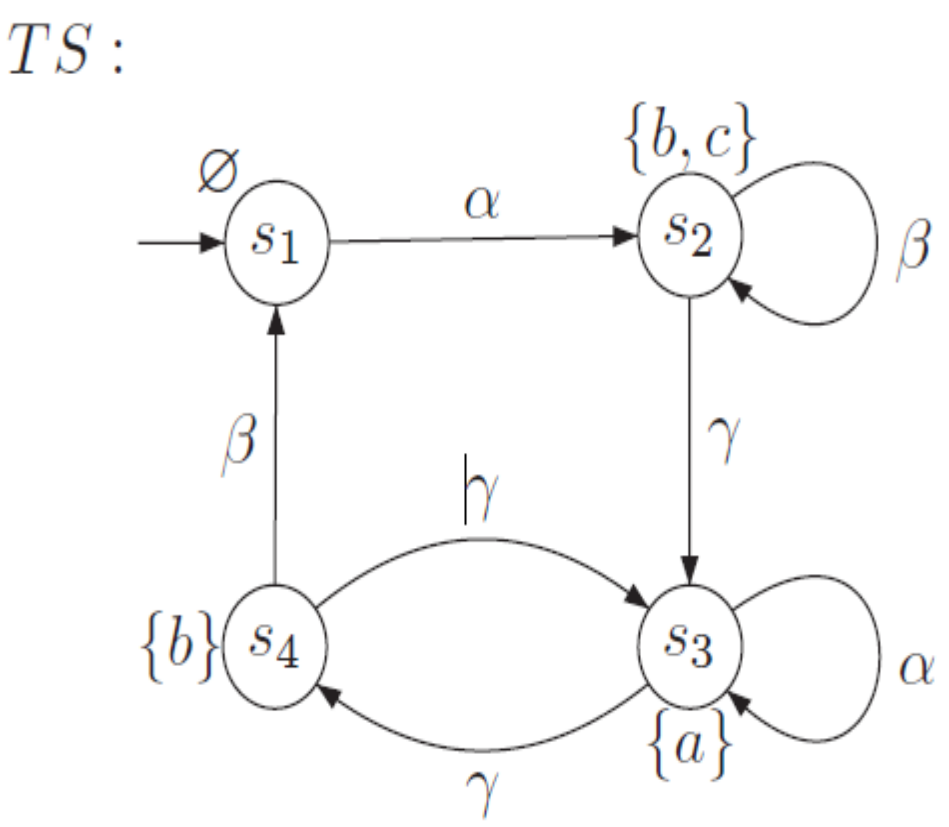
in $TS \otimes A$ for which

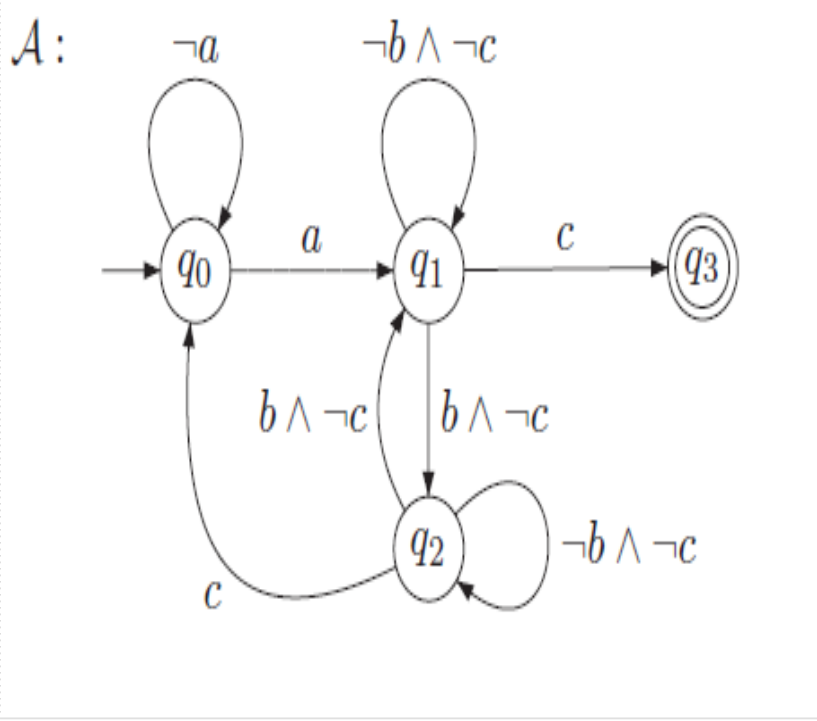
$$q_0 \xrightarrow{L(s_0)} q_1 \xrightarrow{L(s_1)} q_2 \xrightarrow{L(s_1)} \cdots \xrightarrow{L(s_n)} q_{n+1}$$

is a computation of NFA that generates the word

$$trace(\pi) = L(s_0) L(s_1) \cdots L(s_n)$$

Example


TS :
$\varnothing$
s_1
α
$\{b, c\}$
s_2
β
β
γ
γ
$\{b\}$
s_4
s_3
α
$\{a\}$
γ


$\mathcal{A}$:
$\neg a$
$\neg b \wedge \neg c$
q_0
a
q_1
c
q_3
$b \wedge \neg c$
$b \wedge \neg c$
q_2
$\neg b \wedge \neg c$
c

Reducing verification of regular safety property (to what)

- Recall that AP’ =Q. Define

$$P_{inv(A)} := \bigwedge_{q \in F} \neg q$$

- L(A)=MinBadPref(P_{safe}). The following statements are equivalent:

1. $TS \models P_{safe}$;
2. $Traces_{fin}(TS) \cap L(A) = \emptyset$;
3. $TS \otimes A \models P_{inv(A)}$.

Model checking regular safety properties

- In order to check whether $TS \models P_{safe}$, it suffices to check whether $TS \otimes A \models P_{inv(A)}$ (From the above theore
- Reachability analysis: Implement the invariant checking of $P_{inv(A)}$ in $TS \otimes A$.
- If the P_{safe} is refuted, then the algorithm produces a finite path fragment in $TS \otimes A$

$$\langle s_0, q_1 \rangle \langle s_1, q_2 \rangle \cdots \langle s_n, q_{n+1} \rangle$$

where $q_{n+1} \in F$. Correspondingly, we have an initial finite path fragment in TS $s_0 s_1 \cdots s_n$ such that $L(s_0)L(s_1)\cdots L(s_n)$ is accepted by A. So we obtain a counterexample:

$$s_0 s_1 \cdots s_n$$

Algorithm 4 Invariant checking by forward depth-first search

Input: finite transition system TS and propositional formula Φ
Output: "yes" if $TS \models$ "always Φ", otherwise "no" plus a counterexample

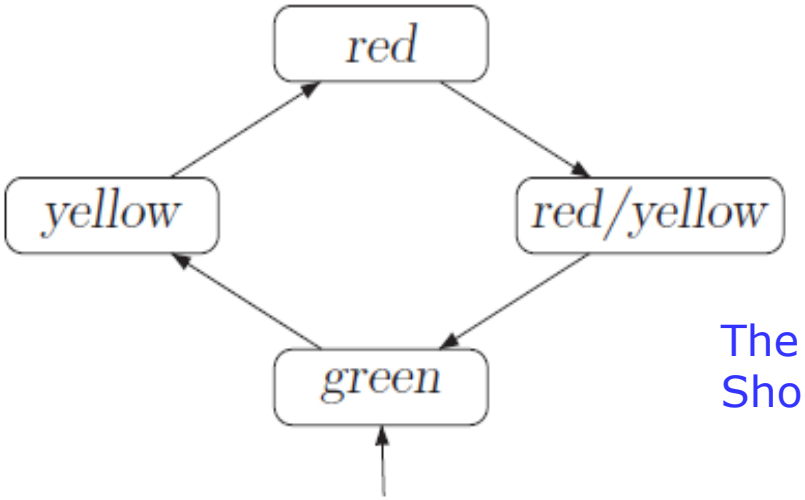
---

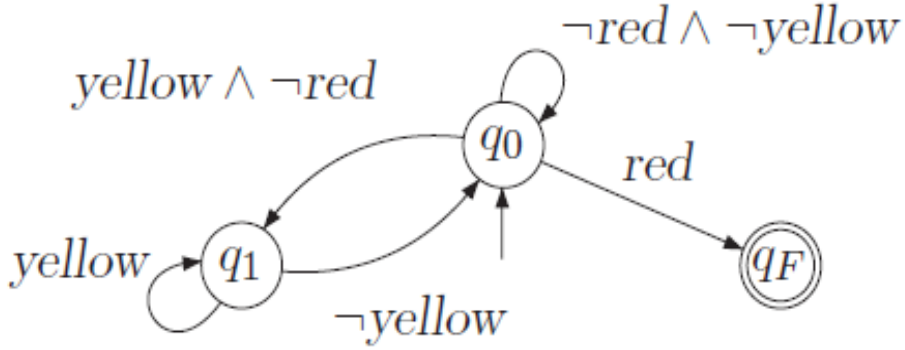
set of states $R := \varnothing$; (* the set of reachable states *)
stack of states $U := \varepsilon$; (* the empty stack *)
bool $b :=$ true; (* all states in R satisfy Φ *)
while $(I \setminus R \neq \varnothing \ \wedge \ b)$ **do**
  let $s \in I \setminus R$; (* chocse an arbitrary initial state not in R *)
  visit(s); (* perform a DFS for each unvisited initial state *)
od
if b **then**
  return("yes") (* $TS \models$ "always Φ" *)
else
  return("no", reverse(U)) (* counterexample arises from the stack content *)
fi

---

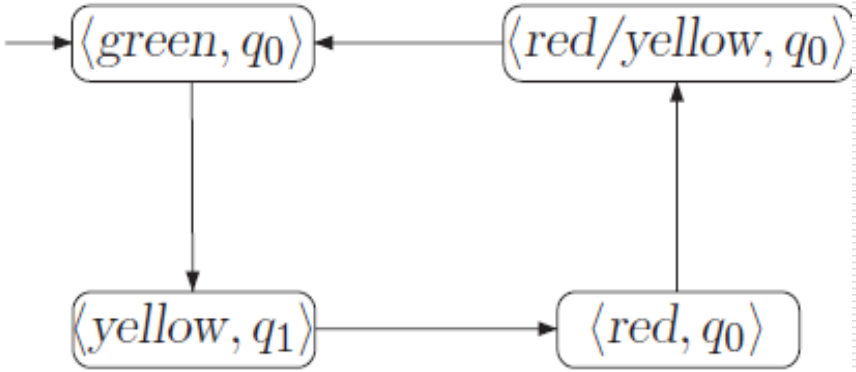
procedure visit (state s)
  $push(s,U)$; (* push s on the stack *)
  $R := R \cup \{ s \}$; (* mark s as reachable *)
  repeat
    $s' := top(U)$;
    if $Post(s') \subseteq R$ **then**
      $pop(U)$;
      $b := b \ \wedge \ (s' \models \Phi)$; (* check validity of Φ in s' *)
    else
      let $s'' \in Post(s') \setminus R$
      $push(s'',U)$;
      $R := R \cup \{ s'' \}$; (* state s'' is a new reachable state *)
    fi
  until $((U = \varepsilon) \ \vee \ \neg b)$
endproc

Example: the German traffic light





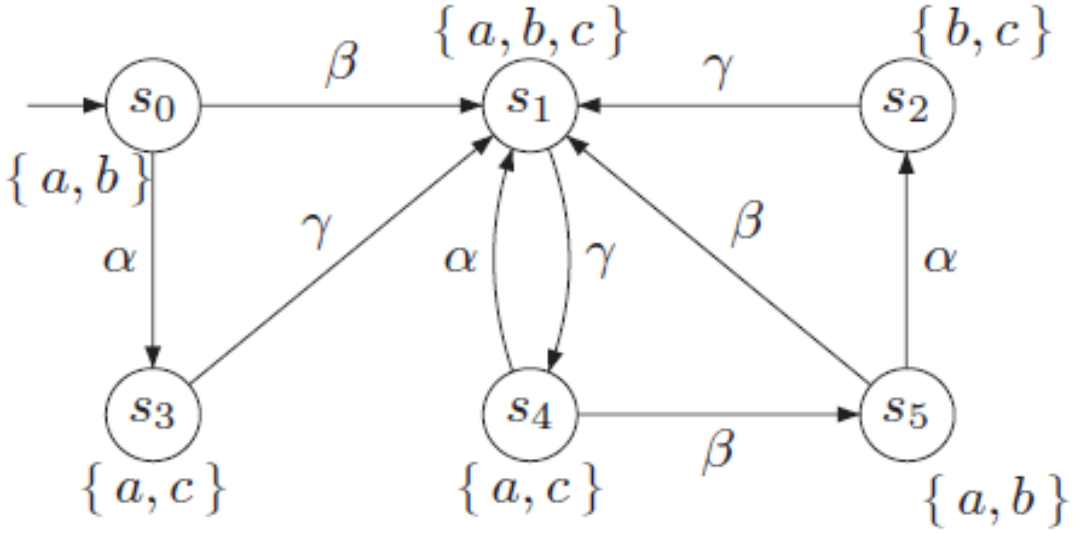
The verified safety property is: a red light phase
Should be immediately preceded by a yellow one.



What if the start state of TS is red instead?

Homework

EXERCISE 4.6. Consider the following transition system TS



and the regular safety property

$P_{safe} =$ "always if a is valid and $b \wedge \neg c$ was valid somewhere before, then a and b do not hold thereafter at least until c holds"